Lab 3 Report: Wall Follower and Safety Controller

Team 11

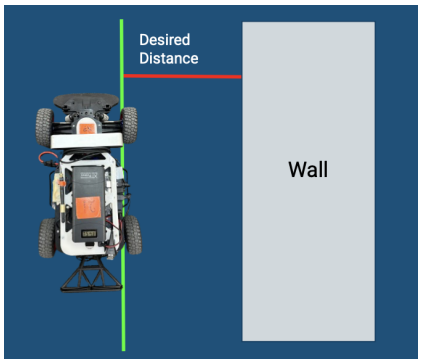
Avishai Jeselsohn
Yuga Patel
Florida Cishesa
Tyler Proctor

6.4200: Robotics Science and Systems

March 13, 2026

1 Introduction

The primary objective of this lab was to implement a robust wall-following algorithm and a safety controller for an autonomous vehicle. These modules not only protect the vehicle against physical damage but also serve as the basis for reactive navigation, establishing safe mobility before the vehicle utilizes complex mapping and environmental visualization to navigate. The wall follower module enables the car to maintain a precise, desired distance from a wall, compensating during operation for variations in vehicle speed and wall orientation. Concurrently, the safety controller acts as a critical fail-safe, continuously monitoring for obstacles to prevent collisions during operation. Furthermore, designing these modules served as a practical introduction to the vehicle's hardware architecture, requiring us to bridge the gap between simulated control logic and real-world sensor integration. The core problem lies in processing noisy sensor data from the LiDAR scanner in real-time and generate stable control. To solve this, both modules rely on filtering raw LiDAR scan data to extract actionable geometric features. The wall-following module feeds this filtered spatial data into a Proportional-Derivative (PD) controller to maintain the cross-track distance, while the safety controller evaluates the data to trigger an emergency brake if an object breaches a predefined minimum distance threshold.



(a) Diagram of wall follower implementation.

(b) Diagram of safety controller implementation.

2 Technical Approach

2.1 Why is the Safety Controller Necesssary?

In the RSS course and in real-world applications, adding safety features to autonomous vehicles is essential. It is both to protect the vehicle itself and everything around in the environment. When the racecar operates at high speeds, it needs a system that prevents collisions. Our main objective for this lab's safety controller was to have the car automatically brake when it detects a nearby wall or obstacle (people, chairs, etc.), safeguarding the racecar from damage while also protecting people and objects in its environment.

2.1.1 Initial Implementation: Fixed Threshold

The safety controller's primary function is real-time obstacle detection and collision avoidance. Two parameters define its behavior: the minimum permissible distance to an obstacle (*stop_distance*) and the angular field of view used to isolate relevant LiDAR returns from extraneous scan data.
The controller processes inputs through a sequential pipeline before publishing any drive commands. Each incoming velocity command is cached and made available to the LiDAR scan callback, which executes the following logic:
Commands received while the vehicle is near-stationary (below a 0.05 m/s velocity threshold) are ignored to prevent false triggering. For each LiDAR scan, the callback extracts range[i] and angle[i] pairs corresponding to detected surfaces. Rays returning NaN or inf values are filtered out as invalid measurements. The remaining data is constrained to a forward-facing angular window of ± 60 degrees, a value selected empirically to balance obstacle coverage with noise rejection. Within this region, the algorithm identifies the minimum range value, corresponding to the nearest detected obstacle. If this minimum range falls below *stop_distance* (defined as 0.5 m), the node executes an emergency stop by publishing zero velocity and zero steering angle.

2.1.2 Updated Approach: Speed-Dependent Linear Threshold

The problem with the fixed threshold is straightforward: a *stop_distance* of 0.5 m might work fine when the car is moving slowly, but at higher speeds the car simply cannot stop in that distance. After testing, we found that the fixed approach failed consistently at faster speeds. The car would overshoot the threshold and hit the obstacle.

To fix this, we first tried gradual braking instead of a hard stop. The idea is to slow the car down as it approaches an obstacle rather than waiting until the last moment. The controller takes the distance to the closest obstacle, subtracts *stop_distance* (so we stop *before* the hard-stop zone, not at the wall), and compares that remaining space to the braking distance the car needs at its current speed. If there is not enough room, the controller scales the commanded speed down proportionally. The closer the obstacle, the slower the car goes. For example, at 2 m/s with $a_{\max} = 4$ m/s^2, the car needs about 0.5 m to come to a full stop.

This was an improvement, but it still did not work well across all speeds because the threshold itself was still fixed. So we took it a step further and made the threshold dynamic. It changes based on how fast the car is currently going. The faster the car moves, the larger the threshold, giving it more room to brake.
We tested the car at different speeds and recorded what threshold distance was needed to stop safely at each one. The data followed a roughly linear trend, so we fit a simple linear model:

$$\text{CurrentVelocity} = m \cdot \text{Threshold} + b \tag{1}$$

$$\text{Threshold} = \frac{\text{CurrentVelocity} - b}{m} \tag{2}$$

where:

- CurrentVelocity is the car's current speed,
- m and b are constants we determined from our test data.

With this model, the we are hoping braking threshold adjusts in real time as the car speeds up or slows down. We have not done extensive tests using this approach, but we are in the process of testing it out if this method works successfully or not.

2.2 Technical Approach for Wall Follower

The wall following module gives the robotic vehicle a way to use the walls of a room to navigate around the perimeter without ever making contact with the wall. We utilize the LiDAR sensor, collecting wall distances at interval angles, and slicing the data to extract the relevant information for upcoming sections of the wall. This is used to find an average distance from the wall, which is

compared to a desired distance parameter passed in. The error between the two is then used by a motor controller to give new speed and steering angle commands to move the vehicle. Although this main functionality remained constant throughout our development life cycle, we had to progress through many iterations to find a final version that was able to complete the different tests we hoped to accomplish.

2.2.1 LiDAR Scan Distance Calculations

As we developed this module, the first step was collecting data from the LiDAR sensor and slicing it. We had to iterate and test to find an efficient range of values to pull from the LiDAR, where we would be able to see a large enough dataset of the wall to correctly map it and follow, but not too much to where there is excess noise or information. We planned to break the LiDAR input into two groups, a side range and a front range. The side range would be used to see the wall that we are planning to follow and move parallel to. In order to do this, we would need to see both directly to the side of the robot and a marginal amount in front, to better detect how the wall would change. Through testing, we were able to arrive at the range of [-$\frac{10\times\pi}{19}$, -$\frac{\pi}{6}$] for the right side, and [$\frac{\pi}{6}$, $\frac{10\times\pi}{19}$] for the left side. The other group of LiDAR scanned data that we utilize is for the front scan, which is within the range of [$-\frac{pi}{50}$, $\frac{pi}{50}$] for the right side, and the inverse for the left side. This scan plays a crucial role of detecting sharper corners and changes in direction of the wall, and allows the robotic vehicle to slow down and make sharper turns. These two slices of the LiDAR data are stored into respective arrays of ranges and angles for each. With this, we are able to calculate the average side or front distance, by first getting x_values and y_values for all the data points. This is done, for x_values, with ranges $\times$ cos(angles), and for y_values, ranges $\times$ sin(angles). We then use linear regression functions under numpy to get slope and y-intercept values from the aggregate x and y values, and calculate distance from the robot.

2.2.2 Controller Module

We decided as a group to utilize a Proportional-Derivative (PD) Controller, and followed the general format utilizing real-time error of the distance, with the formula Distance$_{error}$ = Distance$_{desired}$ + Distance$_{actual}$. This error controlled the speed and steering angle we publish to the robot. We attempted a Proportional-Integral-Derivative (PID) Controller, but due to the shorter time frame of this project and the success we saw with the PD Controller, we decided to move forward with the original design. This led us to test and find optimal k$_p$ and k$_d$ values, landing on k$_p$ = 3.5 and k$_d$ = 2.2. With this, we were able to create the control signal equal to Distance$_{error}\times$ k$_p$ + ΔDistance$_{error}$ $\div$ Δtime$\times$ k$_d$. The steering angle was then set to Multiplied by the negative of the side parameter, this was the steer angle. However, in the case where front scan returns a distance less than or equal to 1.1, the control signal is overridden and a more harsh steering angle of $4.0 \times -$(side), while the speed is capped at 1.8 to ensure

the robot can complete a harder turn in time without hitting the wall. These values are sent as a AckermannDrive message, enabling the robot to perform wall following.

3 Experimental Evaluation

3.1 Safety Controller

For the safety-controller we measured how well the system was operating based on several different testing methods:

- Does the car stop on time when we drop blocks in front of it at random instances?
- Does the car stop if someone steps in front of it?
- Does the car stop within a safe distance from the obstacle at different speeds?

3.1.1 Tests at 1 m/s using 0.5 m threshold

When we tested out the race car safety-controller at 1 m/s with a 0.5 m threshold (distance to stay away from the obstacle) we noticed that the car was sufficiently abiding by all of our testing criteria. For tests 1, one of our team members jumped in front of the car as it was moving, and it successfully stopped. In the second test, we dropped a block in front of the car as it was moving, and the car was able to react in time and come to a complete rest.
When testing detection for obstacles / walls, the car was able to come to a complete rest greater than 0.3 but less than 0.5 m away from the obstacle (walls, chairs, other objects). We also ran the safety controller with the wall follower by making a launch file which allowed both of them to run. When the wall follower failed during the beginning rounds of testing, the safety-controller was effectively able to protect the car from crashing into walls.

3.1.2 Higher Speeds

Using the single threshold approach with an initial velocity of 3m/s, the 0.5m/s threshold was not enough to prevent collision. By manually increasing the breaking-distance to 0.7m/s, the car was able to stop before hitting the obstacle. Using this value, and the 0.5m/s of break-distance for a velocity of 1 m/s, we want to populate the values for the dynamic linear approximation of the Threshold (the break-distance) inside the safety-controller as shown in Fig(2):

$$m = 10/s$$

$$b = 1m/s$$

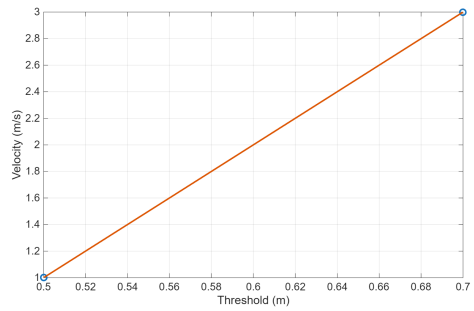


Figure 2: Linear relationship between the current velocity of the Threshold (break-distance) where to deploy safety-controller. The car's velocities were recorded with the corresponding thresholds that allowed timely breaks before collision occured. This linear approximation will be used to dynamically compute the threshold.

where

$$m$$

and

$$b$$

are parameters from eq(1).

3.1.3 Improvements in Future

For upcoming labs and briefings, we hope implement the dynamic Threshold computation, test it on the car, and collect more quantitative data regarding the performance of the software. For this lab, we could have recorded specific quantities of how much the car stopped away from the wall for each speed, and specific distances car stopped at when an obstacle randomly came in front of it.

3.2 Wall Follower

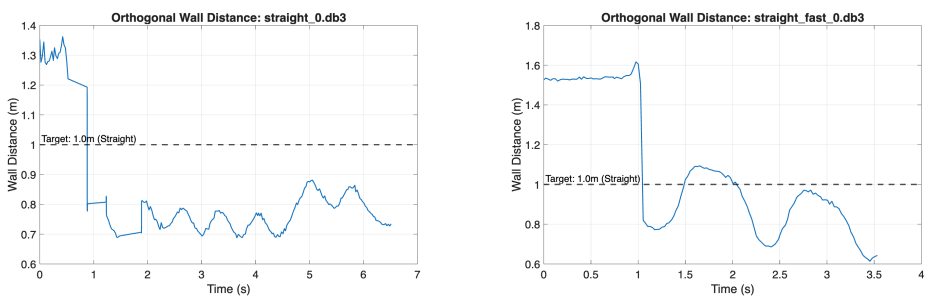
3.2.1 The What, How, and Why

To evaluate our wall follower PD controller, we conducted a series of straight and turning driving tests, measuring the vehicle's ability to maintain a target standoff distance from the wall under varying operational conditions. These tests served to systematically tune the proportional and derivative gains (K_p and K_d) and to quantitatively validate the final controller's stability and accuracy against our performance metric. Success was defined by the vehicle's ability to maintain an actual distance from the wall that remained within a satisfactory, team-defined tolerance of the target distance.
The experimental evaluation was divided into five distinct test cases designed to isolate the effects of velocity and path geometry on the controller's stability. For

each test, the vehicle's real-time distance from the wall was evaluated against the desired distance threshold.

- **Straight Line Performance (Target: 1 meter)**
  - Straight-line driving at 1 m/s.
  - Straight-line driving at 3 m/s.
- **Corner Turn Performance (Target: 0.75 meters)**
  - Turning a single corner for an inner turn at 1 m/s.
  - Turning a single corner for an inner turn at 3 m/s.
  - Turning two corners in succession at 1 m/s.

3.2.2 Results

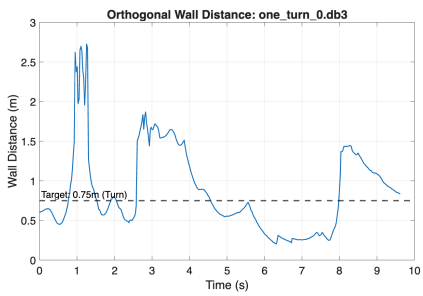


(a) Straight line at 1.0 m/s

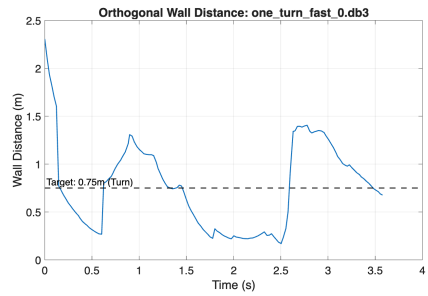
(b) Straight line at 3.0 m/s

Figure 3: Comparison of real distance from wall to target distance from wall over time during straight line tests. The data was extracted from recorded rosbags from the testing.

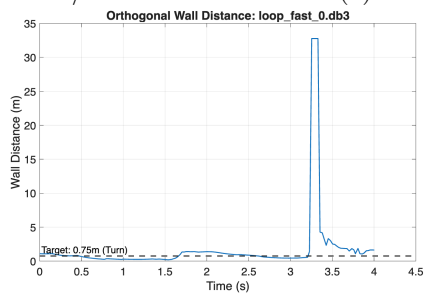
The data we extracted from the straight line tests showed the car's ability to reach the target distance, but it failed to maintain the target distance at a steady rate. As shown in Figure **3a**, after reaching the target distance, the car then proceeded to overshoot and reached 0.8 meters from the wall, maintaining an oscillatory motion within the 0.7-0.8 meter range. This could be attributed to the steady-state error caused by the derivative term in the controller causing it to remain in that range, where increasing the K_p or introducing an integral term could help the car return to the target distance. Additionally, in the faster speed test, shown in Figure **3b**, the car managed to return to the target distance after overshooting, but then proceeded to dip below the target distance and failed to return. The low-pass filter may play a part in this, where the filter weights the previous measurements too heavily, introducing a phase lag into the control loop. Lowering the filter's reliance on past data could prove helpful to minimize the phase lag.



(a) One turn at 1.0 m/s



(b) One turn at 3.0 m/s



(c) Two consecutive turns at 1.0 m/s

Figure 4: Comparison of real distance from wall to target distance from wall over time during turning tests. The data was extracted from recorded rosbags from the testing.

Turning tests demonstrated the vehicle's ability to successfully recover to the target distance after dynamic maneuvers. Anomalous distance spikes in Figures **4a** and **4c** were attributed to a faulty linear regression filtering method temporarily incorporating distant outliers. Despite this sensor noise, the vehicle rapidly re-established accurate distance estimates and successfully converged on the target trajectory. Yet, there existed an oscillatory movement with large amplitudes, which may be fixed by further tuning the K_p or further reducing the phase lag from the low-pass filter.

4 Conclusion

In this lab, we developed and tested both a wall-follower and a safety controller to improve autonomous navigation and obstacle avoidance. The wall-follower's PD design generally kept the robot near the wall, though oscillations and steady-state errors showed that future versions would benefit from adding an integral term.

Complementing the wall-follower, the safety-controller successfully prevented collision in various cases of incoming obstacles. Through experimental trials at varying speeds (1 m/s and 3 m/s), are in the process of implementing a dynamic

threshold calculation to adapt to higher speeds. The integration of both modules showcased the robot's ability to navigate while maintaining spatial awareness and responding appropriately to potential collisions. Future improvements could focus on refining sensor accuracy and implementing adaptive control strategies.

5 Lessons Learned

Avishai - I learned that for an autonomous vehicle, the number one priority is safety. Safety for both the vehicle and the environment around the vehicle. Furthermore, I realize that implementing a controller is not an easy task and takes a lot of iteration and testing in a real world environment in order to perfect it, and even then it won't be perfect. Using a PD controller to drive a car needs to be couple with other controller algorithms to better stabilize the car and allow it to drive smoothly around any terrain and with any obstacle. I also learned how to interact with the vehicle itself and got more attuned to the intricacies of git. I think for future labs I will do a better job at communicating my thought process and a better job at hearing what the other team members think and trying to implement their ideas. I think I didn't work as much I had hoped for on this lab and I want to improve that moving forward.

Yuga - I learned a lot about how to put code onto the hardware itself. Understanding how different docker systems from my computer and robot work were fascinating to understand. I also got a lot more familiarity using github, and understanding how we can stay organized by making different branches for using and running our code. It was also really interesting to think about potential implications of what code on physical systems does. Thinking about different edge cases and making sure we account for all of them. It was also a lot different from running everything in simulation, and accounting for different factors like friction and steering offsets taught me a different style of thinking.

Florida - From this lab, I realized the importance of strong communication in a team project. I also gained a lot of experience with Git and Ros, and to manage them on different platforms as I am working with teammates. I particularly enjoyed learning about the different Git commands and how to properly use github because it has always been an area of confusion in previous classes. I also enjoyed thinking about different strategies for improving the performance of our safety-controller, as it was yielding unexpected behavior than what my teammate observed in simulation. Being able to communicate my thoughts and brainstorm ideas for improvement was great team effort. For future labs, I want to get better at communicating responsibility and the role of teamwork such that it makes our team more efficient at making progress.

Tyler - From this lab, I learned a lot about how programs can perform vastly different on simulation versus when being tested on the real robot and interacting with a real environment. I also realized how important it is to begin testing early, and use real testing as the benchmark for how well the program or code is performing. I think if too much time is spent on sim or just examining the

code, there will be limited time to actually see how the robot performs and make the smaller adjustments that lead to a successful project. This also stems from communication, as the team can work to set a better schedule up that ensures adequate time is allocated for testing. Additionally, I learned that there is a lot more that goes into a project than just having well-performing code. I saw how important it is to be able to present a final product, discuss the progress of the project, and see the broader implications of the work. I think I want to improve how I approach these topics, and make sure I keep these ideas in mind while I'm designing or building the deliverables.